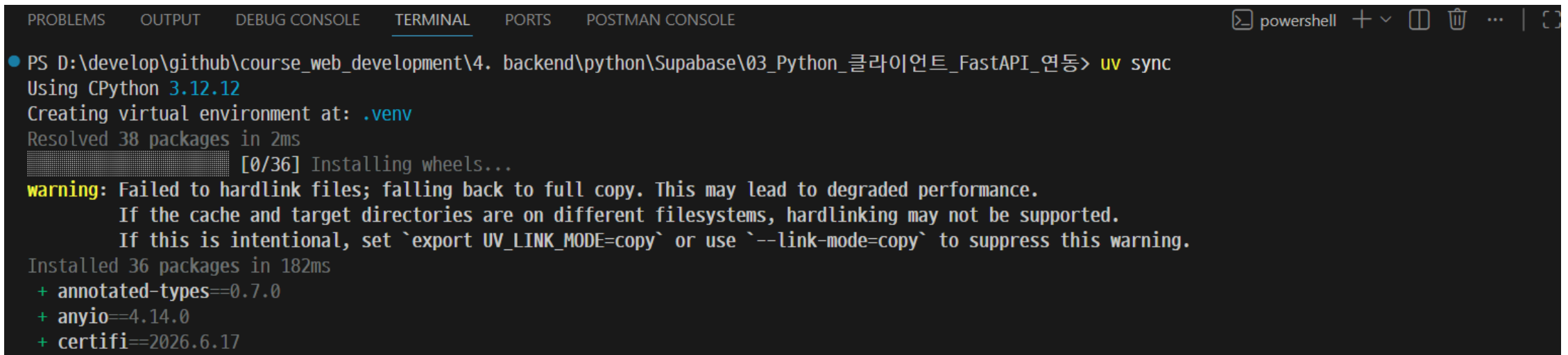


Supabase Python 클라이언트와 FastAPI 연동

1. 가상환경 생성

```
# pyproject.toml과 uv.lock 기준으로 의존성 설치  
uv sync
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE powershell + - [ ] [ ] ... | [ ]  
● PS D:\develop\github\course_web_development\4. backend\python\Supabase\03_Python_클라이언트_FastAPI_연동> uv sync  
Using CPython 3.12.12  
Creating virtual environment at: .venv  
Resolved 38 packages in 2ms  
[0/36] Installing wheels...  
warning: Failed to hardlink files; falling back to full copy. This may lead to degraded performance.  
If the cache and target directories are on different filesystems, hardlinking may not be supported.  
If this is intentional, set `export UV_LINK_MODE=copy` or use `--link-mode=copy` to suppress this warning.  
Installed 36 packages in 182ms  
+ annotated-types==0.7.0  
+ anyio==4.14.0  
+ certifi==2026.6.17
```

2. 환경 변수로 API 키 관리

왜 환경 변수를 써야 하는가

해킹의 위험이 발생함

```
# 하드코딩 - GitHub에 올리면 즉시 노출
supabase = create_client(
    "https://abcdef.supabase.co",
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
)
```

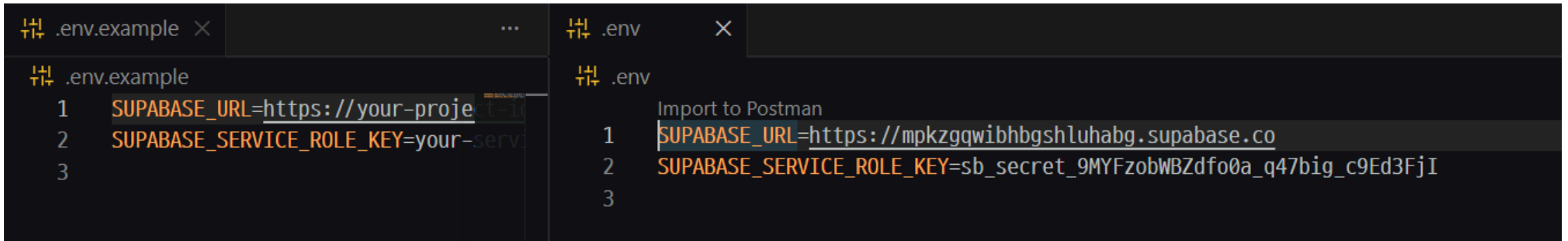
중요한 정보(키)를 코드에서 확인이 불가능함 -> 해킹 위험이 없음

```
# 환경 변수 - 코드에 키가 없음
import os
from dotenv import load_dotenv

load_dotenv() # .env 파일 읽기
supabase = create_client(
    os.environ["SUPABASE_URL"], os.environ["SUPABASE_SERVICE_ROLE_KEY"],
)
```

.env.example → .env 파일 구성

Supabase의 Settings를 통해 API Key 확인



```
.env.example
1 SUPABASE_URL=https://your-project-id.supabase.co
2 SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
3

.env
1 Import to Postman
2 SUPABASE_URL=https://mpkzgqwibhbgshluhabg.supabase.co
3 SUPABASE_SERVICE_ROLE_KEY=sb_secret_9MYFzobWBZdfo0a_q47big_c9Ed3FjI
```

3. Supabase 클라이언트 vs SQL 비교

Supabase 클라이언트는 SQL을 Python 메서드 체이닝으로 표현합니다.

SQL 문법	Supabase 클라이언트
<code>SELECT * FROM app_users</code>	<code>.table("app_users").select("*")</code>
<code>SELECT id, username FROM ...</code>	<code>.select("id, username")</code>
<code>WHERE id = 'xxx'</code>	<code>.eq("id", "xxx")</code>
<code>WHERE role != 'system'</code>	<code>.neq("role", "system")</code>
<code>ORDER BY created_at DESC</code>	<code>.order("created_at", desc=True)</code>
<code>ORDER BY created_at ASC</code>	<code>.order("created_at", desc=False)</code>

SQL 문법	Supabase 클라이언트
<code>LIMIT 10</code>	<code>.limit(10)</code>
<code>INSERT INTO ... VALUES (...)</code>	<code>.insert({...})</code>
<code>INSERT ... RETURNING *</code>	<code>.insert({...})</code> (기본적으로 생성된 행 반환)
<code>UPDATE ... SET ... WHERE</code>	<code>.update({...}).eq("id", id)</code>
<code>DELETE FROM ... WHERE</code>	<code>.delete().eq("id", id)</code>

마지막에 반드시 `.execute()` 를 붙여야 실행됩니다.

```
# execute() 없으면 실행 안 됨!
result = supabase.table("app_users").select("*").execute()
#                                     ↑ 반드시 필요
```

3. Supabase 클라이언트 테스트

The screenshot shows a Jupyter Notebook titled "Supabase 클라이언트 테스트.ipynb" in a VS Code environment. The Explorer sidebar on the left shows the file structure with an "OUTLINE" section containing the following items:

- M 라이브러리 임포트
- M .env 파일의 데이터 환경변수에 등록
- M 환경변수에 등록된 데이터 변수에 저장
- M Supabase 클라이언트 생성
- M 간단한 조회로 연결 테스트

The notebook content is divided into two sections:

Supabase 클라이언트 생성

```
supabase: Client = create_client(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY)
```

[4] ✓ 0.4s Python

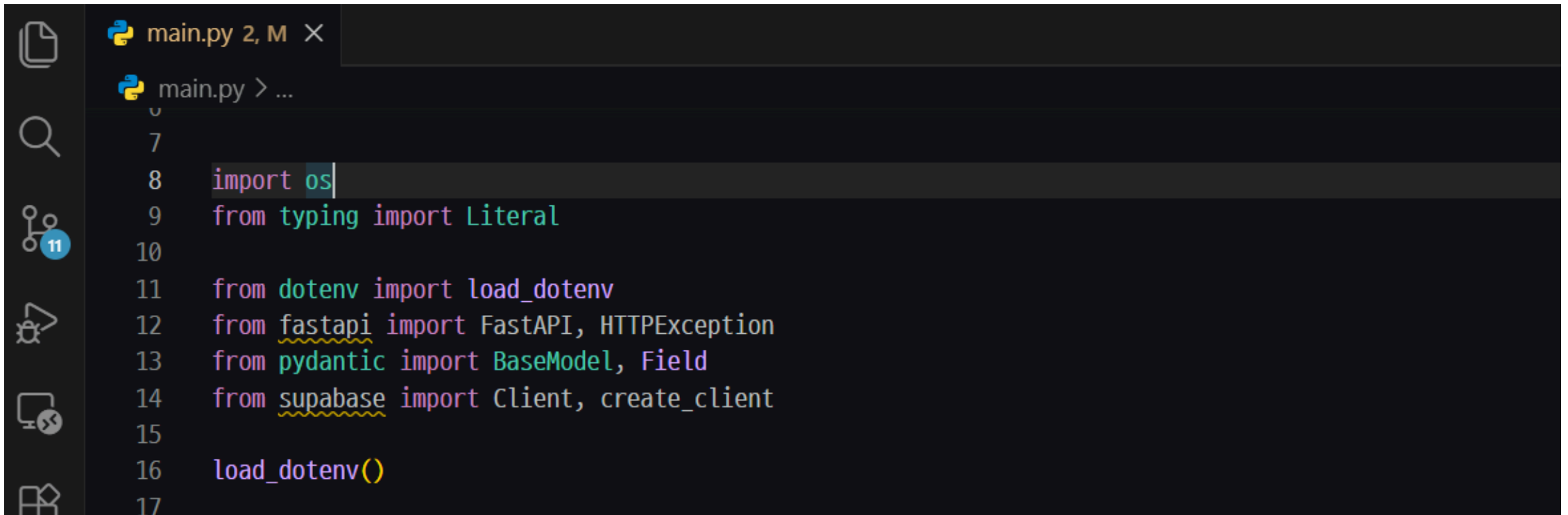
간단한 조회로 연결 테스트

```
result = supabase.table("app_users").select("*").limit(1).execute()
print(result.data) # [] 또는 데이터 리스트 출력되면 연결 성공
```

[6] ✓ 1.6s Python

... [{"id": "4c460602-eaf4-4e6e-a1d2-781843690bf7", "username": "kim", "display_name": "김학생", "created_at": "2026-06-17T11:24:57.807583+00:00"}]

4. FastAPI 엔드포인트 연결



```
main.py 2, M ×  
main.py > ...  
7  
8 import os  
9 from typing import Literal  
10  
11 from dotenv import load_dotenv  
12 from fastapi import FastAPI, HTTPException  
13 from pydantic import BaseModel, Field  
14 from supabase import Client, create_client  
15  
16 load_dotenv()  
17
```

Pydantic 모델 데이터를 딕셔너리로 변환

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    username: str
    display_name: str

data = UserCreate(username="kim", display_name="김 학생")

# .model_dump()로 딕셔너리로 변환
payload = data.model_dump() # {"username": "kim", "display_name": "김 학생"}

result = supabase.table("app_users").insert(payload).execute()
```

사용자 생성

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI()

class UserCreate(BaseModel):
    username: str = Field(..., min_length=2, max_length=30)
    display_name: str = Field(..., min_length=1, max_length=50)

@app.post("/users", status_code=201)
def create_user(data: UserCreate):
    ...
```

사용자 목록 조회

```
@app.get("/users")
def list_users():
    result = (
        supabase.table("app_users")
        .select("*")
        .order("created_at", desc=True)
        .execute()
    )
    return result.data # 빈 리스트여도 그냥 반환
```

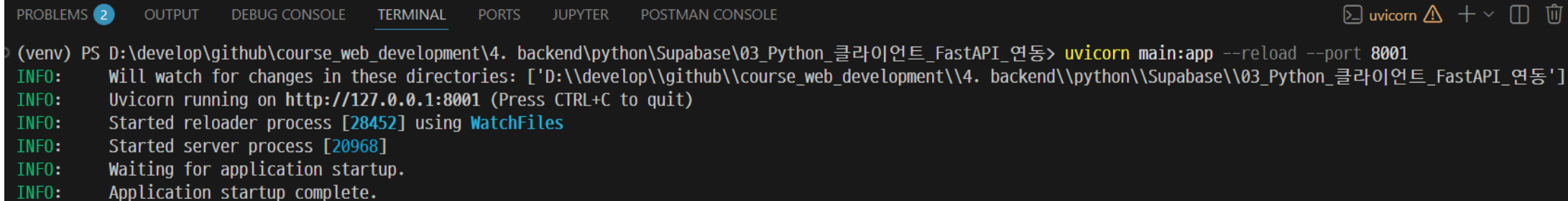
대화방 메시지 조회

```
@app.get("/conversations/{conversation_id}/messages")
def list_messages(conversation_id: str):
    result = (
        supabase.table("messages")
        .select("*")
        .eq("conversation_id", conversation_id)
        .order("created_at") # 오래된 것부터 (기본 ASC)
        .execute()
    )
    return result.data
```

5. 서버 실행 및 Swagger 실습

서버 실행

```
# 1. .env 파일에 실제 키 입력  
# 2. 서버 실행  
uvicorn main:app --reload --port 8001
```



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The command prompt is '(venv) PS D:\develop\github\course_web_development\4. backend\python\Supabase\03_Python_클라이언트_FastAPI_연동>'. The command entered is 'uvicorn main:app --reload --port 8001'. The output shows several INFO messages: 'Will watch for changes in these directories: [...]', 'Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)', 'Started reloader process [28452] using WatchFiles', 'Started server process [20968]', 'Waiting for application startup.', and 'Application startup complete.'.

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER POSTMAN CONSOLE  
(venv) PS D:\develop\github\course_web_development\4. backend\python\Supabase\03_Python_클라이언트_FastAPI_연동> uvicorn main:app --reload --port 8001  
INFO: Will watch for changes in these directories: ['D:\\develop\\github\\course_web_development\\4. backend\\python\\Supabase\\03_Python_클라이언트_FastAPI_연동']  
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)  
INFO: Started reloader process [28452] using WatchFiles  
INFO: Started server process [20968]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
|
```

Swagger UI 실행 (<http://localhost:8001/docs>)

POST /users → Body에 {"username": "hong", "display_name": "홍길동"} →
Execute

POST /users Create User

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "username": "hong",  "display_name": "홍길동"}

```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "hong",
    "display_name": "홍길동"
  }'
```



Request URL

```
http://127.0.0.1:8001/users
```

Server response

Code

Details

201

Response body

```
{
  "id": "721114b7-4314-4c88-bb02-76246f72cd2a",
  "username": "hong",
  "display_name": "홍길동",
  "created_at": "2026-06-21T03:17:04.093741+00:00"
}
```



Download

Response headers

```
content-length: 138
content-type: application/json
date: Sun, 21 Jun 2026 03:17:04 GMT
server: uvicorn
```

GET /users 목록 확인

GET

/users List Users

^

Parameters

Cancel

No parameters

Execute

Responses

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/users' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8001/users
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
[
  {
    "id": "721114b7-4314-4c88-bb02-76246f72cd2a",
    "username": "hong",
    "display_name": "홍길동",
    "created_at": "2026-06-21T03:17:04.093741+00:00"
  },
  {
    "id": "6507e74d-00a1-4ce3-8a34-55bd39e473b0",
    "username": "cho1",
    "display_name": "홍학생1",
    "created_at": "2026-06-21T03:07:11.314707+00:00"
  },
]
```

GET /users 목록 확인을 통해 user_id 복사

POST /conversations → Body에 {"user_id": "복사한-UUID", "title": "첫 대화"}

→ Execute → id 복사

The screenshot shows a REST client interface for a POST request to the endpoint `/conversations`. The interface includes a header bar with the method `POST` and the endpoint `/conversations`, along with a 'Create Conversation' label and an expand/collapse icon. Below the header, there are two tabs: 'Parameters' and 'Request body'. The 'Parameters' tab is currently selected, showing 'No parameters'. To the right of the 'Parameters' tab are 'Cancel' and 'Reset' buttons. Below the 'Parameters' tab, the 'Request body' section is visible, marked as 'required'. It features a dropdown menu set to 'application/json'. The 'Request body' section has two sub-tabs: 'Edit Value' and 'Schema'. The 'Edit Value' tab is active, displaying a JSON object:

```
{  "user_id": "721114b7-4314-4c88-bb02-76246f72cd2a",  "title": "첫 대화"}
```

. At the bottom of the interface is a large blue 'Execute' button.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/conversations' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "721114b7-4314-4c88-bb02-76246f72cd2a",
    "title": "첫 대화"
  }'
```

Request URL

`http://127.0.0.1:8001/conversations`

Server response

Code

Details

201

Response body

```
{
  "id": "b13f8105-6076-42e3-84c9-7f5a029f0416",
  "user_id": "721114b7-4314-4c88-bb02-76246f72cd2a",
  "title": "첫 대화",
  "created_at": "2026-06-21T03:21:47.286252+00:00"
}
```



Download

Response headers

```
content-length: 163
content-type: application/json
date: Sun, 21 Jun 2026 03:21:47 GMT
server: uvicorn
```

POST /conversations/{id}/messages → Body에 {"role": "user", "content": "안녕 !"} → Execute

POST /conversations/{conversation_id}/messages Create Message

Parameters

Cancel

Reset

Name	Description
conversation_id ★ required string <i>{path}</i>	b13f8105-6076-42e3-84c9-7f5a029f0416

Request body required

application/json

Edit Value | Schema

```
{  "role": "user",  "content": "안녕"}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/conversations/b13f8105-6076-42e3-84c9-7f5a029f0416/messages' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "role": "user",
    "content": "안녕"
  }'
```



Request URL

```
http://127.0.0.1:8001/conversations/b13f8105-6076-42e3-84c9-7f5a029f0416/messages
```

Server response

Code

Details

201

Response body

```
{
  "id": "21f8ef07-ca4a-4bcd-8dbc-e0d6f7248ba1",
  "conversation_id": "b13f8105-6076-42e3-84c9-7f5a029f0416",
  "role": "user",
  "content": "안녕",
  "created_at": "2026-06-21T03:26:44.415061+00:00"
}
```



Download

Response headers

```
content-length: 183
content-type: application/json
date: Sun, 21 Jun 2026 03:26:44 GMT
server: uvicorn
```

POST /conversations/{id}/messages → Body에 {"role": "assistant",
"content": "안녕하세요!"} → Execute

POST

/conversations/{conversation_id}/messages

Create Message

^

Parameters

Cancel

Reset

Name	Description
conversation_id ★ required string (path)	b13f8105-6076-42e3-84c9-7f5a029f0416

Request body required

application/json ▼

Edit Value | Schema

```
{  "role": "assistant",  "content": "안녕하세요"}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/conversations/b13f8105-6076-42e3-84c9-7f5a029f0416/messages' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "role": "assistant",
    "content": "안녕하세요"
  }'
```

Request URL

```
http://127.0.0.1:8001/conversations/b13f8105-6076-42e3-84c9-7f5a029f0416/messages
```

Server response

Code

Details

201

Response body

```
{
  "id": "9589f757-2d82-4e73-808f-fa7fd2824aa6",
  "conversation_id": "b13f8105-6076-42e3-84c9-7f5a029f0416",
  "role": "assistant",
  "content": "안녕하세요",
  "created_at": "2026-06-21T03:28:15.74174+00:00"
}
```



Download

Response headers

```
content-length: 196
content-type: application/json
date: Sun, 21 Jun 2026 03:28:15 GMT
server: uvicorn
```

GET /conversations/{id}/messages → 메시지 2개 확인

GET

/conversations/{conversation_id}/messages

List Messages

^

Parameters

Cancel

Name	Description
conversation_id * required string (path)	b13f8105-6076-42e3-84c9-7f5a029f0416

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/conversations/b13f8105-6076-42e3-84c9-7f5a029f0416/messages' \
  -H 'accept: application/json'
```



Request URL

```
http://127.0.0.1:8001/conversations/b13f8105-6076-42e3-84c9-7f5a029f0416/messages
```

Server response

Code

Details

200

Response body

```
[
  {
    "id": "21f8ef07-ca4a-4bcd-8dbc-e0d6f7248ba1",
    "conversation_id": "b13f8105-6076-42e3-84c9-7f5a029f0416",
    "role": "user",
    "content": "안녕",
    "created_at": "2026-06-21T03:26:44.415061+00:00"
  },
  {
    "id": "9589f757-2d82-4e73-808f-fa7fd2824aa6",
    "conversation_id": "b13f8105-6076-42e3-84c9-7f5a029f0416",
    "role": "assistant",
    "content": "안녕하세요",
    "created_at": "2026-06-21T03:28:15.74174+00:00"
  }
]
```

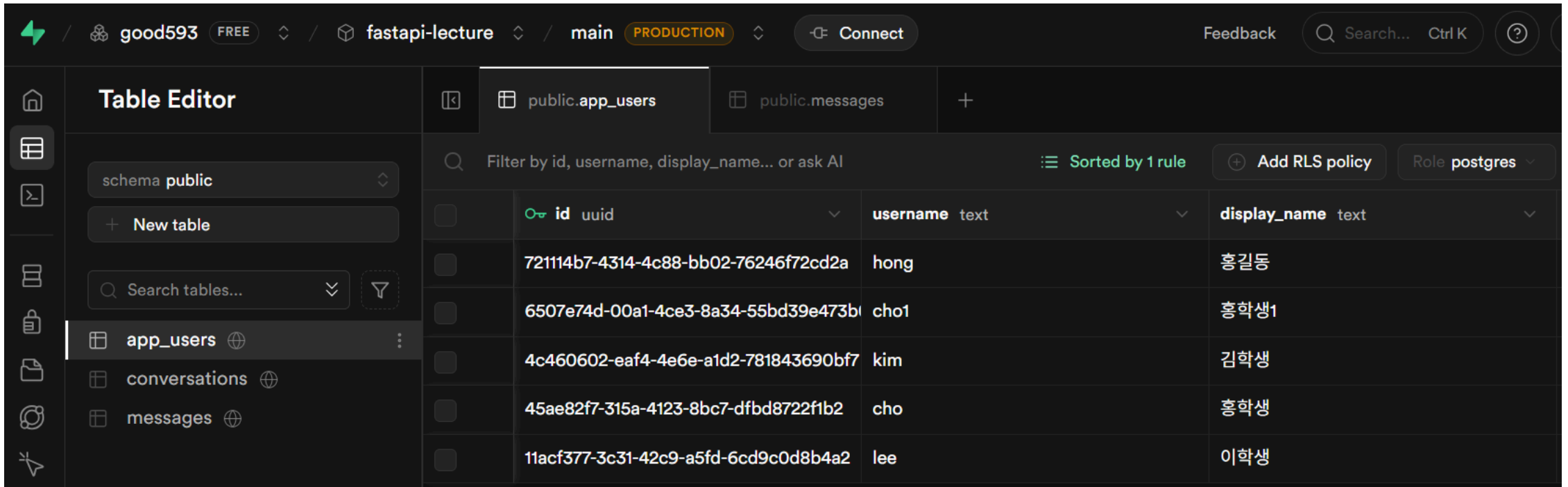


Download

Response headers

```
content-length: 382
content-type: application/json
date: Sun, 21 Jun 2026 03:29:29 GMT
server: uvicorn
```

Supabase Table Editor에서 저장된 데이터 직접 확인



The screenshot shows the Supabase Table Editor interface. The top navigation bar includes the Supabase logo, project name 'good593' (FREE), database name 'fastapi-lecture', and environment 'main' (PRODUCTION). A 'Connect' button is visible. The left sidebar contains navigation icons for home, table editor, SQL editor, schema browser, and other tools. The main area is titled 'Table Editor' and shows the 'public.app_users' table selected. The table has three columns: 'id' (uuid), 'username' (text), and 'display_name' (text). The table is sorted by 1 rule. The data rows are as follows:

id	username	display_name
721114b7-4314-4c88-bb02-76246f72cd2a	hong	홍길동
6507e74d-00a1-4ce3-8a34-55bd39e473b1	cho1	홍학생1
4c460602-eaf4-4e6e-a1d2-781843690bf7	kim	김학생
45ae82f7-315a-4123-8bc7-dfbd8722f1b2	cho	홍학생
11acf377-3c31-42c9-a5fd-6cd9c0d8b4a2	lee	이학생